Post-Quantum Cryptography for Resource-Constrained IoT and Edge Devices: A No-Std Rust Implementation Framework

Prabakaran Kannan
Research Scholar, Department of Computer Science and Engineering
National Institute of Technology Puducherry
Email: cs23d2005@nitpy.ac.in

***Abstract*—The proliferation of Internet of Things (IoT) devices in critical infrastructure—smart meters, railway signaling systems, and border surveillance sensors—creates an urgent need for quantum-resistant cryptography that operates within severe resource constraints. We present Q-EDGE-OS, a comprehensive post-quantum cryptographic framework implemented in pure no-std Rust, targeting microcontrollers with as little as 256 KB flash and 64 KB RAM. Our framework provides production-grade implementations of ML-KEM-768 (FIPS 203), ML-DSA-65 (FIPS 204), and FN-DSA-512 for key encapsulation and digital signatures, alongside five novel constructions: (1) a post-quantum mesh networking handshake protocol achieving mutual authentication and key agreement over LoRa links with 255-byte MTU constraints; (2) ML-DSA-65-based secure boot with Merkle tree anti-rollback protection; (3) hardware-rooted device identity binding via Physical Unclonable Functions (PUFs) and eFUSE; (4) Shamir secret sharing over GF(2^8) for threshold key recovery on constrained nodes; and (5) remote attestation with post-quantum integrity proofs. We demonstrate the framework on three target architectures—ARM Cortex-M7 (STM32H743), ARM Cortex-M33 with TrustZone (STM32U585), and RISC-V (SiFive FE310)—and evaluate it through three real-world applications: energy smart metering, SIL-4 railway signaling, and mesh-networked border sensors. Our NTT-optimized implementations achieve ML-KEM-768 encapsulation in 1.2 ms on Cortex-M7 at 480 MHz and complete the full mesh handshake within 15 ms, demonstrating that NIST-standard post-quantum cryptography is practical for deployment on resource-constrained IoT devices today.**

***Index Terms*—Post-quantum cryptography, IoT security, ML-KEM, ML-DSA, Falcon, embedded systems, mesh networking, secure boot, Rust, no_std, resource-constrained devices**

I. INTRODUCTION

The quantum computing threat to classical public-key cryptography poses a particularly acute challenge for IoT and edge devices deployed in critical infrastructure. Unlike enterprise systems that can be patched remotely, IoT devices face unique constraints: limited computational resources (often <512 KB flash, <128 KB RAM), bandwidth-constrained communication links (LoRa at 0.3–50 kbps), real-time deadlines (SIL-4 railway signaling requires <10 ms response), and extended deployment lifetimes of 15–25 years that overlap with projected timelines for cryptographically relevant quantum computers [1].

The NIST Post-Quantum Cryptography standardization process has produced three primary standards—ML-KEM (FIPS 203) [2], ML-DSA (FIPS 204) [3], and SLH-DSA (FIPS 205) [4]—alongside FN-DSA (Falcon) [5] as an additional signature standard. However, reference implementations are primarily designed for desktop and server environments, with C-language codebases that present challenges for memory-safe embedded deployment.

This paper presents Q-EDGE-OS, a comprehensive post-quantum cryptographic framework implemented entirely in no-std Rust, designed for deployment on resource-constrained IoT and edge devices. Our key contributions are:

1) **Pure no-std PQC implementations**: Constant-time, heap-free implementations of ML-KEM-768, ML-DSA-65, and FN-DSA-512 with NTT optimizations for both Kyber ($q = 3329$) and Dilithium ($q = 8{,}380{,}417$) fields.
2) **Post-quantum mesh handshake**: A mutual authentication and key agreement protocol for LoRa mesh networks, combining ML-KEM-768 key encapsulation with ML-DSA-65 signatures and HKDF-SHA3-256 key derivation, operating within a 255-byte MTU.
3) **PQ-secure boot chain**: ML-DSA-65-based firmware verification with Merkle tree anti-rollback counters and hardware-rooted identity binding via PUF/eFUSE.
4) **Threshold key recovery**: Shamir secret sharing over GF(2^8) enabling (t, n)-threshold key recovery for devices deployed in hostile environments.
5) **Remote attestation**: Post-quantum integrity proofs combining device identity, firmware measurement, and ML-DSA-65 signatures for fleet-wide trust verification.
6) **Multi-architecture validation**: Deployment on ARM Cortex-M7, Cortex-M33 (TrustZone), and RISC-V with comprehensive benchmarks across three real-world applications.

The remainder of this paper is organized as follows. Section II reviews post-quantum cryptographic primitives and

TABLE I
TARGET PLATFORM SPECIFICATIONS

Parameter	STM32H743	STM32U585	FE310
Architecture	Cortex-M7	Cortex-M33	RV32IMAC
Clock (MHz)	480	160	320
Flash (KB)	2048	2048	16384
RAM (KB)	1024	786	256
TrustZone	No	Yes	No
FPU	DP	SP	No
Crypto Accel.	AES/HASH	AES/PKA	None

embedded constraints. Section III presents the Q-EDGE-OS framework architecture. Section IV details the core cryptographic implementations. Section V describes the post-quantum mesh handshake protocol. Section VI covers secure boot and device identity. Section VII presents threshold key recovery. Section VIII describes remote attestation. Section IX provides performance evaluation. Section XI discusses related work, and Section XII concludes.

II. BACKGROUND AND PRELIMINARIES

A. NIST Post-Quantum Standards

The NIST PQC standardization process [6] culminated in the publication of three Federal Information Processing Standards in 2024:

ML-KEM (FIPS 203) [2] is a key encapsulation mechanism based on the Module Learning With Errors (MLWE) problem. ML-KEM-768 provides NIST Security Level 3 with a 1,184-byte public key, 1,088-byte ciphertext, and 32-byte shared secret.

ML-DSA (FIPS 204) [3] is a digital signature scheme based on Module-LWE and Module-SIS problems. ML-DSA-65 achieves NIST Security Level 3 with a 1,952-byte public key and 3,309-byte signature.

FN-DSA (Falcon) [5] produces compact signatures (≈666 bytes at Level 1) using NTRU lattices with fast Fourier sampling, making it attractive for bandwidth-constrained IoT links.

B. IoT Resource Constraints

Table I summarizes the target platform capabilities that constrain our design decisions. The primary challenges are:

- **Memory**: ML-KEM-768 requires ≈20 KB working memory for NTT computations; ML-DSA-65 requires ≈45 KB for signing.
- **Bandwidth**: LoRa MTU of 255 bytes requires multi-fragment transmission for PQ key material.
- **Latency**: Safety-critical applications (railway SIL-4) require cryptographic operations to complete within hard real-time deadlines.
- **Power**: Battery-powered border sensors must minimize cryptographic computation to achieve multi-year lifetimes.

C. Threat Model

We consider adversaries with access to a cryptographically relevant quantum computer (CRQC) capable of executing Shor's algorithm [7] to break RSA and ECC, and Grover's algorithm [8] to accelerate symmetric key search. Our threat model encompasses:

- **Harvest-now-decrypt-later (HNDL)**: Adversaries recording encrypted IoT telemetry for future quantum decryption.
- **Physical access**: Devices deployed in unprotected environments (border sensors, smart meters) subject to physical tampering.
- **Supply chain attacks**: Compromised firmware injected during manufacturing or OTA updates.
- **Network-level attacks**: Man-in-the-middle attacks on mesh communication links.

III. FRAMEWORK ARCHITECTURE

Q-EDGE-OS is organized as a collection of Rust crates following the embedded Rust ecosystem conventions [9], each compiled with `no_std` to eliminate operating system dependencies. The architecture comprises seven crates:

1) **q-crypto**: Core PQC primitives (ML-KEM-768, ML-DSA-65, FN-DSA-512), SHA3 hash functions, AEAD ciphers, and NIST-compliant DRBG.
2) **q-mesh**: Post-quantum mesh networking with handshake protocol, encrypted sessions, LoRa radio drivers, and group trust management.
3) **q-boot**: Secure boot verification with anti-rollback protection and firmware integrity checking.
4) **q-identity**: Hardware-rooted device identity using PUF challenge-response and eFUSE binding.
5) **q-recover**: Threshold secret sharing for distributed key recovery.
6) **q-attest**: Remote attestation protocol for fleet-wide device integrity verification.
7) **q-hal**: Hardware Abstraction Layer providing unified APIs for STM32H743, STM32U585, and SiFive FE310.

Design principles. All cryptographic operations are implemented with constant-time arithmetic to prevent timing side-channels. Memory is stack-allocated exclusively—no heap allocations occur in any crate. The trait-based abstraction layer (`Kem`, `Signer`, `Hash`, `Aead`, `CryptoRng`, `Kdf`, `Xof`) enables compile-time algorithm selection without runtime polymorphism overhead.

IV. CORE CRYPTOGRAPHIC IMPLEMENTATIONS

A. Field Arithmetic and NTT

The foundation of our ML-KEM and ML-DSA implementations is optimized field arithmetic for two distinct prime fields:

Kyber field ($q = 3329$): Elements are represented as `u16` values in the Montgomery domain with $R = 2^{16}$. Barrett reduction is used for multiplication with the precomputed constant $\lfloor 2^{32}/q \rfloor$. The Number Theoretic Transform (NTT) operates on 256-element polynomials with precomputed zeta values (primitive 512th root of unity modulo 3329).

Algorithm 1 ML-KEM-768 Encapsulation (Embedded)

Require: Public key pk, randomness source $\mathcal{R}$
Ensure: Ciphertext ct, shared secret ss
1: $m \leftarrow \mathcal{R}.\text{fill}(32)$ {DRBG random bytes}
2: $(\bar{K}, r) \leftarrow \text{SHA3-512}(m \| \text{SHA3-256}(pk))$
3: $\hat{u}, v \leftarrow \text{Encrypt}(pk, m, r)$ {NTT-based}
4: $ct \leftarrow \text{Compress}(\hat{u}) \| \text{Compress}(v)$
5: $ss \leftarrow \text{SHA3-256}(\bar{K} \| \text{SHA3-256}(ct))$
6: **return** (ct, ss)

Dilithium field ($q = 8{,}380{,}417$): Elements use `u32` representation. Montgomery multiplication uses $R = 2^{32}$ with the precomputed inverse $q^{-1} \bmod R$. The NTT uses distinct zeta tables for the Dilithium field.

Both NTT implementations use the Cooley-Tukey butterfly for forward transforms and Gentleman-Sande for inverse transforms, with all zeta values precomputed at compile time to minimize runtime memory allocation.

B. ML-KEM-768 Implementation

Our ML-KEM-768 implementation follows FIPS 203 [2] precisely, with embedded-specific optimizations:

Key optimizations for embedded targets include:

- **In-place NTT**: All polynomial transformations operate in-place, avoiding temporary buffer allocation. Peak working memory is 20 KB.
- **Streaming compression**: Ciphertext compression is interleaved with NTT computation to reduce peak stack usage.
- **Constant-time comparison**: Decapsulation uses bitwise OR accumulation for implicit rejection, preventing timing side-channels.

C. ML-DSA-65 Implementation

ML-DSA-65 (FIPS 204) [3] is implemented with the following parameters: module dimension $k = 6, \ell = 5$, coefficient bound $\eta = 4$, challenge weight $\tau = 49$, and dropping bits $d = 13$.

The signing procedure uses rejection sampling with the Fiat-Shamir-with-aborts paradigm [10]. Our implementation tracks the number of rejection loop iterations and exposes this as a diagnostic metric for entropy quality monitoring.

D. FN-DSA-512 Implementation

FN-DSA-512 (Falcon) [5] is included for bandwidth-constrained scenarios where ML-DSA-65's 3,309-byte signatures exceed link MTU budgets. FN-DSA-512 produces ≈666-byte signatures at NIST Level 1, enabling single-fragment transmission over LoRa links.

The implementation uses NTRU lattice-based trapdoor sampling with fast Fourier arithmetic. Key generation is computationally expensive (≈150 ms on Cortex-M7) but is a one-time operation performed during device provisioning.

E. SHA3 and SHAKE Functions

The Keccak-f[1600] permutation is implemented with lane-complementing optimization, achieving 12.5 cycles/byte on Cortex-M7. We provide SHA3-256, SHA3-384, SHA3-512, SHAKE128, and SHAKE256 as building blocks. The SHAKE extendable output functions are critical for ML-KEM's noise sampling and ML-DSA's mask generation.

F. AEAD Ciphers

Two authenticated encryption modes are supported:

- **AES-256-GCM**: Used on platforms with hardware AES acceleration (STM32H7, STM32U5). GF(2^{128}) multiplication uses the Karatsuba method with precomputed H tables.
- **ChaCha20-Poly1305**: Software-only alternative for platforms without AES hardware (FE310). Quarter-round operations leverage the RISC-V rotate instructions where available.

G. Random Number Generation

Cryptographic randomness is generated using an NIST SP 800-90A [11] compliant HMAC-DRBG with SHA-256, seeded from hardware entropy sources. Entropy quality is continuously monitored per SP 800-90B [12] with repetition count and adaptive proportion health tests. The DRBG supports reseeding at configurable intervals (default: every 4096 requests or 65,536 bytes).

V. Post-Quantum Mesh Handshake Protocol

A. Protocol Design

The Q-EDGE-OS mesh handshake protocol establishes authenticated, encrypted sessions between IoT nodes communicating over bandwidth-constrained LoRa links. The protocol combines ML-KEM-768 for key encapsulation, ML-DSA-65 for mutual authentication, and HKDF-SHA3-256 for key derivation.

B. Message Fragmentation

LoRa frames are limited to 255 bytes MTU (with a 40-byte Q-EDGE-OS header, yielding 215 bytes payload per frame). ML-KEM-768 public keys (1,184 bytes) and ML-DSA-65 signatures (3,309 bytes) must be fragmented across multiple frames. The handshake protocol uses a sliding window fragment reassembly mechanism with:

- **Fragment header**: 4 bytes (message ID, fragment index, total fragments, flags).
- **Timeout**: Configurable per-fragment timeout (default 5s) with exponential backoff.
- **Duplicate detection**: Bitmap-based tracking to handle LoRa's inherent retransmissions.

The complete 3-phase handshake requires 28–32 LoRa frames depending on radio configuration (SF7–SF12), completing in approximately 15 ms of CPU time (dominated by the ML-KEM and ML-DSA operations) plus radio transmission time.

Algorithm 2 PQ Mesh Handshake Protocol

Require: Initiator I with $(pk_I^{\text{sig}}, sk_I^{\text{sig}})$
Require: Responder R with $(pk_R^{\text{sig}}, sk_R^{\text{sig}})$
Ensure: Shared session keys $(k_{\text{tx}}, k_{\text{rx}})$
// Phase 1: Ephemeral KEM key exchange
I: $(ek, dk) \leftarrow$ ML-KEM-768.KeyGen()
$I \rightarrow R$: INIT$\|ek\|pk_I^{\text{sig}}$ {Fragmented}
R: $(ct, ss) \leftarrow$ ML-KEM-768.Encaps(ek)
R: $\sigma_R \leftarrow$ ML-DSA-65.Sign$(sk_R^{\text{sig}}, ct\|ek)$
$R \rightarrow I$: RESP$\|ct\|\sigma_R\|pk_R^{\text{sig}}$
// Phase 2: Mutual authentication
I: Verify$(\sigma_R, pk_R^{\text{sig}}, ct\|ek)$
I: $ss \leftarrow$ ML-KEM-768.Decaps(dk, ct)
I: $\sigma_I \leftarrow$ ML-DSA-65.Sign$(sk_I^{\text{sig}}, ss\|ct)$
$I \rightarrow R$: AUTH$\|\sigma_I$
R: Verify$(\sigma_I, pk_I^{\text{sig}}, ss\|ct)$
// Phase 3: Key derivation
$k_{\text{master}} \leftarrow$ HKDF-SHA3-256$(ss,$ "q-mesh-v1")
$(k_{\text{tx}}, k_{\text{rx}}) \leftarrow$ HKDF-Expand$(k_{\text{master}},$ "session", $64)$

C. Encrypted Session Management

After handshake completion, all subsequent frames are encrypted using AES-256-GCM or ChaCha20-Poly1305 (selected at compile time based on hardware support). Each session maintains:

- Transmit and receive keys $(k_{\text{tx}}, k_{\text{rx}})$ with directional separation.
- 64-bit frame counter with replay window (default: 1024 frames).
- Session lifetime with configurable re-keying interval.

D. Group Trust Management

The mesh protocol supports hierarchical trust with five levels (Untrusted, Basic, Verified, Trusted, Admin) and four roles (Member, Router, Gateway, Controller). Trust decisions are based on:

- **Certificate chain verification**: ML-DSA-65 certificate chains rooted in a fleet-level root of trust.
- **Behavioral scoring**: Nodes that forward messages reliably accrue trust; misbehaving nodes are demoted.
- **Quorum-based promotion**: Trust level upgrades require attestation from $\geq$3 existing trusted nodes.

E. LoRa Radio Integration

The framework includes native drivers for the Semtech SX1276/SX1278 LoRa transceivers with support for EU868, US915, and AS923 frequency plans. The driver provides:

- Configurable spreading factor (SF7–SF12) and bandwidth (125–500 kHz).
- Automatic duty cycle management per regional regulations.
- RSSI and SNR monitoring for adaptive spreading factor selection.
- Listen-before-talk carrier sensing for collision avoidance.

Algorithm 3 PQ Secure Boot Verification

Require: Firmware image F, signature σ_F, root public key pk_{root}
Require: Anti-rollback counter c_{current}, counter in image c_F
Ensure: Boot decision: ACCEPT or REJECT
$h_F \leftarrow$ SHA3-256(F)
if $\neg$ ML-DSA-65.Verify$(pk_{\text{root}}, h_F\|c_F, \sigma_F)$ **then**
  return REJECT {Signature invalid}
end if
if $c_F < c_{\text{current}}$ **then**
  return REJECT {Rollback detected}
end if
$c_{\text{current}} \leftarrow c_F$ {Update monotonic counter}
Verify Merkle proof for firmware block integrity
return ACCEPT

VI. Secure Boot and Device Identity

A. ML-DSA-65 Secure Boot

Q-EDGE-OS implements a post-quantum secure boot chain that verifies firmware integrity before execution:

The root public key (pk_{root}, 1,952 bytes for ML-DSA-65) is stored in one-time-programmable (OTP) memory or eFUSE. The anti-rollback counter uses a Merkle tree structure where each firmware version corresponds to a leaf, enabling efficient proof of version ordering without linear counter storage.

B. Hardware-Rooted Identity

Device identity is bound to hardware through two mechanisms:

PUF-based identity: On supported platforms, Silicon Physical Unclonable Functions (PUFs) [13] generate device-unique challenge-response pairs. The PUF response is used as entropy input to derive a device-specific ML-DSA-65 key pair during first boot. The challenge-response is repeatable but physically unclonable, binding the cryptographic identity to the specific silicon die.

eFUSE binding: On platforms without PUF support, a 256-bit device secret is programmed into one-time-programmable eFUSE memory during manufacturing. This secret, combined with the firmware hash, derives the device identity key pair.

Both mechanisms ensure that cryptographic identity cannot be cloned to another physical device, preventing impersonation attacks in mesh networks.

C. A/B Partition OTA Updates

Secure Over-the-Air (OTA) firmware updates use an A/B partition scheme:

1) New firmware is downloaded to the inactive partition.
2) ML-DSA-65 signature is verified against the OTA signing key (which chains to pk_{root}).
3) Anti-rollback counter is checked.
4) Boot flag is set to the new partition.
5) On next boot, the secure boot chain verifies the new firmware.

Algorithm 4 Shamir Secret Sharing – Split

Require: Secret S (byte array), threshold t, total shares n
Ensure: Shares $\{(x_i, y_i)\}_{i=1}^{n}$
1: **for** each byte s_j of S **do**
2: Generate random polynomial $f_j(x) = s_j + a_1 x + \cdots + a_{t-1}x^{t-1}$
3: where $a_1, \ldots, a_{t-1} \leftarrow \mathcal{R}.\text{fill}(t-1)$
4: **for** $i = 1$ to n **do**
5: $y_{i,j} \leftarrow f_j(i)$ evaluated in GF(2^8)
6: **end for**
7: **end for**
8: **return** $\{(i, \{y_{i,j}\}_j)\}_{i=1}^{n}$

6) If verification fails, automatic rollback to the previous partition occurs.

This ensures that a failed or tampered OTA update cannot brick the device.

VII. Threshold Key Recovery

IoT devices deployed in hostile environments (border sensors, remote infrastructure) may suffer physical damage or loss. Q-EDGE-OS implements Shamir's Secret Sharing [14] over GF(2^8) for distributed key recovery.

A. GF(2^8) Arithmetic

Operations in GF(2^8) use the irreducible polynomial $x^8 + x^4 + x^3 + x + 1$ (0x11B). Multiplication is implemented via log/exp tables (512 bytes total), providing constant-time $O(1)$ field operations.

B. Recovery Protocol

Key recovery proceeds when t of n shares are collected from surviving neighboring nodes:

1) The replacement device broadcasts a RECOVERYREQUEST message signed by the fleet controller.
2) Each share-holding node verifies the controller signature and responds with its share, encrypted under the requesting device's ephemeral ML-KEM-768 public key.
3) Upon collecting t shares, the device reconstructs the secret using Lagrange interpolation over GF(2^8):

$$S = \sum_{i=1}^{t} y_i \prod_{\substack{j=1 \\ j \neq i}}^{t} \frac{x_j}{x_j \oplus x_i}$$

where $\oplus$ denotes XOR (addition in GF(2^8)) and division uses the log/exp tables.

Typical configurations use $(3, 5)$ or $(4, 7)$ threshold schemes, balancing availability against compromise tolerance.

VIII. Remote Attestation

Q-EDGE-OS implements a remote attestation protocol that enables a fleet controller to verify the integrity of deployed devices:

Attestation report. Each device generates an attestation report containing:

TABLE II
CRYPTOGRAPHIC PRIMITIVE PERFORMANCE (MILLISECONDS)

Operation	M7 480 MHz	M33 160 MHz	RV32 320 MHz
ML-KEM-768 KeyGen	0.8	2.4	3.1
ML-KEM-768 Encaps	1.2	3.5	4.6
ML-KEM-768 Decaps	1.1	3.3	4.3
ML-DSA-65 KeyGen	2.1	6.3	8.2
ML-DSA-65 Sign	4.8	14.4	18.7
ML-DSA-65 Verify	2.3	6.9	9.0
FN-DSA-512 KeyGen	150	450	585
FN-DSA-512 Sign	8.5	25.5	33.2
FN-DSA-512 Verify	1.5	4.5	5.9
SHA3-256 (1 KB)	0.08	0.24	0.31
AES-256-GCM (1 KB)	0.03	0.09	N/A
ChaCha20-Poly1305 (1 KB)	0.05	0.15	0.12

- Device identity (PUF-derived ML-DSA-65 public key fingerprint).
- Firmware version and SHA3-256 hash of the executing image.
- Boot chain measurements (secure boot verification log).
- Hardware configuration hash (peripheral state, clock configuration).
- Monotonic timestamp from hardware RTC.

The report is signed using the device's PUF-derived ML-DSA-65 private key, creating a cryptographic binding between the device identity, its current firmware state, and the attestation moment.

Fleet-level verification. The controller maintains a database of expected firmware hashes and device public keys. Attestation responses that deviate from expected values trigger quarantine procedures: the suspect device is isolated from mesh routing, and an OTA re-provisioning sequence is initiated.

IX. Performance Evaluation

A. Cryptographic Primitive Benchmarks

Table II presents benchmark results for the core cryptographic operations across the three target platforms. All measurements are averaged over 1,000 iterations with caches warm.

B. Memory Footprint

Table III shows the memory requirements for each framework component.

The full framework fits within 172 KB flash and 82.5 KB peak RAM—well within the capabilities of all three target platforms. A minimal configuration (ML-KEM-768 + secure boot) requires only 64 KB flash and 31 KB RAM, enabling deployment on even more constrained devices.

C. Mesh Handshake Performance

Table IV breaks down the mesh handshake timing across the three protocol phases.

The CPU computation time (14 ms) is negligible compared to LoRa transmission time. At SF7 (highest data rate, shortest range), the complete handshake requires approximately 2.1

TABLE III
MEMORY FOOTPRINT BY COMPONENT

Component	Flash (KB)	RAM (KB)
q-crypto (all primitives)	98	48
ML-KEM-768 only	32	20
ML-DSA-65 only	38	45
FN-DSA-512 only	28	35
SHA3 + SHAKE	8	1.6
AEAD (both)	12	2
q-mesh (handshake + session)	24	16
q-boot (verify only)	14	8
q-identity (PUF + eFUSE)	6	2
q-recover (Shamir SSS)	4	1.5
q-attest	8	4
q-hal (per platform)	18	3
Full stack	**172**	**82.5**
Minimal (KEM + boot)	**64**	**31**

TABLE IV
MESH HANDSHAKE PHASE TIMING (CORTEX-M7, 480 MHZ)

Phase	CPU Time	Bytes
Phase 1: KEM KeyGen + Encaps	2.0 ms	2,272
Phase 2: Sign + Verify (both)	11.9 ms	6,618
Phase 3: HKDF key derivation	0.1 ms	0
Fragment overhead (headers)	—	512
Total (CPU)	**14.0 ms**	**9,402**
Total (LoRa SF7 125kHz)	**≈2.1 s**	—
Total (LoRa SF12 125kHz)	**≈47 s**	—

seconds. At SF12 (lowest data rate, maximum range), it extends to approximately 47 seconds—acceptable for initial device pairing in border sensor deployments where handshakes occur infrequently.

D. Application Case Studies

1) Smart Energy Metering: The smart meter application (STM32H743 target) performs:

- Secure meter reading transmission every 15 minutes using AES-256-GCM with PQ-established session keys.
- Daily ML-DSA-65 signed attestation reports to the utility backend.
- Monthly key rotation via mesh handshake re-establishment.

The cryptographic overhead is 6.2 ms per reading cycle (0.0007% of the 15-minute interval), with negligible impact on power consumption.

2) Railway Signaling (SIL-4): The railway signaling application (STM32U585 with TrustZone) meets SIL-4 safety requirements [15], [16]:

- Secure boot completes in 18 ms (well within the 100 ms boot budget).
- Message authentication (HMAC-SHA3-256 with PQ-established keys) adds 0.3 ms per signaling message.
- TrustZone isolation separates safety-critical signaling logic in the Secure World from communication stack in the Non-Secure World.
- Dual-channel verification: both channels independently verify signatures before accepting state changes.

TABLE V
COMPARISON WITH EXISTING EMBEDDED PQC IMPLEMENTATIONS

Feature	Ours	pqm4	liboqs	wolfSSL
Language	Rust	C/ASM	C	C
no_std	Yes	Yes	No	Partial
Heap-free	Yes	Yes	No	No
Memory safe	Yes	No	No	No
ML-KEM	768	512–1024	All	768
ML-DSA	65	44–87	All	65
FN-DSA	512	No	All	No
Mesh protocol	Yes	No	No	No
Secure boot	PQ	No	No	No
Key recovery	SSS	No	No	No
Attestation	PQ	No	No	No
Multi-arch	3	M4 only	Desktop	M4+

3) Border Sensor Mesh Network: The border sensor application (SiFive FE310 + LoRa) demonstrates the mesh networking capabilities:

- Initial handshake with 3–5 neighboring sensors using the PQ mesh protocol.
- Event-driven encrypted alerts (motion detection, tamper alarm) forwarded via multi-hop mesh routing.
- $(3, 5)$ Shamir secret sharing of the device key across neighboring nodes for recovery.
- Monthly remote attestation via gateway relay to the command center.
- Target: 5-year battery life with solar charging, achieved by duty-cycling the radio (0.1% active time) and using lightweight HMAC-SHA3 for routine messages, reserving full PQ signatures for key management events.

E. Comparison with Existing Approaches

Table V compares Q-EDGE-OS with existing PQC implementations for embedded systems.

Q-EDGE-OS is the first framework to provide an integrated, memory-safe PQC stack with mesh networking, secure boot, key recovery, and remote attestation specifically designed for resource-constrained IoT devices.

X. SECURITY ANALYSIS

A. Quantum Security Guarantees

All key encapsulation and signature operations achieve NIST Security Level 3 (ML-KEM-768, ML-DSA-65) or Level 1 (FN-DSA-512). The security of ML-KEM-768 reduces to the hardness of the Module-LWE problem with parameters $(k = 3, q = 3329, \eta_1 = 2, \eta_2 = 2)$. The security of ML-DSA-65 reduces to Module-LWE and Module-SIS with parameters $(k = 6, \ell = 5, q = 8380417)$.

B. Side-Channel Resistance

All cryptographic operations are implemented with constant-time guarantees:

- **No secret-dependent branches**: All conditional operations use bitwise masking rather than branch instructions.

- **No secret-dependent memory access**: Array indexing does not depend on secret values; where necessary, constant-time table lookups are used.
- **Implicit rejection**: ML-KEM decapsulation uses the FO transform with implicit rejection to prevent chosen-ciphertext attacks.

C. Mesh Protocol Security

The mesh handshake achieves:

- **Key agreement**: The shared secret is computationally indistinguishable from random under the IND-CCA2 security of ML-KEM-768.
- **Mutual authentication**: Both parties verify ML-DSA-65 signatures on the KEM ciphertext, binding authentication to the key exchange.
- **Forward secrecy**: Ephemeral KEM key pairs ensure that compromise of long-term signing keys does not reveal past session keys.
- **Replay protection**: Per-session nonces and frame counters prevent replay attacks.

D. Secure Boot Trust Chain

The secure boot chain's security relies on:

- **Root of trust**: The ML-DSA-65 root public key in OTP/eFUSE is immutable post-provisioning.
- **Anti-rollback**: The monotonic Merkle counter prevents downgrade attacks to firmware versions with known vulnerabilities.
- **Hardware binding**: PUF-derived keys cannot be extracted or cloned, preventing firmware execution on unauthorized hardware.

E. Limitations

- FN-DSA-512 provides only NIST Level 1 security; for long-term deployments (>15 years), ML-DSA-65 at Level 3 is recommended despite the larger signatures.
- The mesh handshake transmits PQ key material in cleartext during Phase 1, enabling traffic analysis (though not key compromise). A pre-shared key mode could mitigate this at the cost of deployment complexity.
- PUF reliability degrades with temperature extremes; environmental compensation must be calibrated per deployment.

XI. Related Work

PQC on embedded platforms. The pqm4 project [17] provides optimized C/assembly implementations of PQC schemes for ARM Cortex-M4, focusing on algorithmic benchmarking rather than integrated application support. Alkim et al. demonstrated Cortex-M4 optimizations for lattice-based schemes, achieving cycle counts within 3× of our Cortex-M7 results when normalized for clock speed. Our work extends beyond primitive optimization to provide a complete security stack.

IoT PQC surveys. Misoczki et al. [18] surveyed PQC suitability for IoT, identifying memory constraints and key/signature sizes as primary challenges. Our framework validates these findings and demonstrates practical solutions through careful memory management and protocol-level compression.

Mesh networking security. The Noise Protocol Framework [19] provides authenticated key exchange patterns but relies on X25519/X448 (vulnerable to quantum attack). Q-EDGE-OS adapts Noise-like handshake patterns to PQC primitives while accommodating the larger key/ciphertext sizes through fragmentation.

LoRaWAN security. The LoRaWAN specification [20] uses AES-128-CTR for payload encryption with pre-shared root keys. This provides no forward secrecy and is vulnerable to HNDL attacks. Our mesh protocol establishes ephemeral PQ session keys with forward secrecy over LoRa physical links.

Rust for embedded security. The Rust embedded ecosystem [9] has grown significantly, with the `no_std` attribute enabling deployment on bare-metal microcontrollers. Rust's ownership model and borrow checker provide memory safety guarantees at compile time, eliminating entire classes of vulnerabilities (buffer overflows, use-after-free) that plague C-based embedded cryptographic implementations.

NIST migration guidance. NIST SP 800-227 [21] and NSA CNSA 2.0 [22] provide transition timelines, recommending PQC deployment by 2030 for systems protecting information with 20+ year confidentiality requirements—precisely the category of IoT devices addressed by Q-EDGE-OS.

XII. Conclusion and Future Work

We have presented Q-EDGE-OS, a comprehensive post-quantum cryptographic framework for resource-constrained IoT and edge devices, implemented entirely in no-std Rust. The framework provides production-grade implementations of ML-KEM-768, ML-DSA-65, and FN-DSA-512, alongside five novel constructions: a PQ mesh handshake protocol, ML-DSA-65 secure boot, hardware-rooted identity, threshold key recovery, and remote attestation with PQ integrity proofs.

Our evaluation across three target architectures (Cortex-M7, Cortex-M33, RISC-V) and three real-world applications (smart metering, railway signaling, border sensors) demonstrates that NIST-standard post-quantum cryptography is practical for IoT deployment today, with the full framework requiring only 172 KB flash and 82.5 KB RAM.

Future work includes: (1) formal verification of the mesh handshake protocol using the Tamarin prover; (2) hardware acceleration of NTT operations using the Cortex-M7 DSP instructions; (3) integration with the QBITEL Bridge platform's crypto agility framework for hybrid classical-PQ migration; and (4) field trials with commercial smart meter and railway signaling deployments.

References

[1] M. Mosca, "Cybersecurity in an era with quantum computers: Will we be ready?" *IEEE Security & Privacy*, vol. 16, no. 5, pp. 38–41, 2018.

[2] National Institute of Standards and Technology, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," NIST, Federal Information Processing Standards Publication FIPS 203, 2024, https://doi.org/10.6028/NIST.FIPS.203.

[3] ——, "Module-Lattice-Based Digital Signature Standard," NIST, Federal Information Processing Standards Publication FIPS 204, 2024, https://doi.org/10.6028/NIST.FIPS.204.

[4] ——, "Stateless Hash-Based Digital Signature Standard," NIST, Federal Information Processing Standards Publication FIPS 205, 2024, https://doi.org/10.6028/NIST.FIPS.205.

[5] P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang, "Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU," in *NIST PQC Submission*, 2020, round 3 Finalist.

[6] D. Moody, L. Chen, D. Dodson, Y.-K. Liu, R. Perlner, A. Regenscheid, and D. Smith-Tone, "Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process," NIST, Internal Report NISTIR 8413, 2022.

[7] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997.

[8] L. K. Grover, "A Fast Quantum Mechanical Algorithm for Database Search," *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pp. 212–219, 1996.

[9] Rust Embedded Working Group, "The Embedded Rust Book," 2024, https://docs.rust-embedded.org/book/.

[10] V. Lyubashevsky, "Lattice Signatures Without Trapdoors," *Advances in Cryptology – EUROCRYPT 2012*, vol. 7237, pp. 738–755, 2012.

[11] National Institute of Standards and Technology, "Recommendation for Random Number Generation Using Deterministic Random Bit Generators," NIST SP 800-90A Rev. 1, 2015.

[12] ——, "Recommendation for the Entropy Sources Used for Random Bit Generation," NIST SP 800-90B, 2018.

[13] G. E. Suh and S. Devadas, "Physical Unclonable Functions for Device Authentication and Secret Key Generation," in *44th ACM/IEEE Design Automation Conference*, 2007, pp. 9–14.

[14] A. Shamir, "How to Share a Secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[15] International Electrotechnical Commission, *Functional safety of electrical/electronic/programmable electronic safety-related systems*, Std. IEC 61 508, 2010.

[16] European Committee for Electrotechnical Standardization, *Railway applications – Communication, signalling and processing systems – Safety related electronic systems for signalling*, Std. EN 50 129, 2018.

[17] E. Alkim, Y. A. Bilgin, M. Cenk, and F. Gérard, "Cortex-M4 Optimizations for R, M, LWE Schemes," in *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2020.

[18] R. Misoczki, J. Ding, X. Dong, and Z. Zhang, "Post-Quantum Cryptography for IoT: A Survey," *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–34, 2023.

[19] T. Perrin, "The Noise Protocol Framework," in *Revision 34*, 2018, https://noiseprotocol.org/noise.html.

[20] LoRa Alliance, "LoRaWAN Specification v1.0.4," in *Technical Specification*, 2022, https://lora-alliance.org/resource-hub.

[21] National Institute of Standards and Technology, "Recommendations for Transition to Post-Quantum Cryptography," NIST, Special Publication 800-227, 2025.

[22] National Security Agency, "Commercial National Security Algorithm Suite 2.0," NSA, Tech. Rep., 2022, https://media.defense.gov/2022/Sep/07/2003071834/-1/-1/0/CSA_CNSA_2.0_ALGORITHMS_.PDF.